Digital Systems & Microcontrollers

Lecture 1: Binary Number Systems

How two voltage states become numbers, codes, memory and computation

0 ↔ LOW voltage

1 ↔ HIGH voltage

Core flow: motivation → conversions → complements → signed subtraction → BCD & codes

Warmup: where digital systems quietly run the world

Each example reduces messy real-world signals into numbers that can be stored and processed.

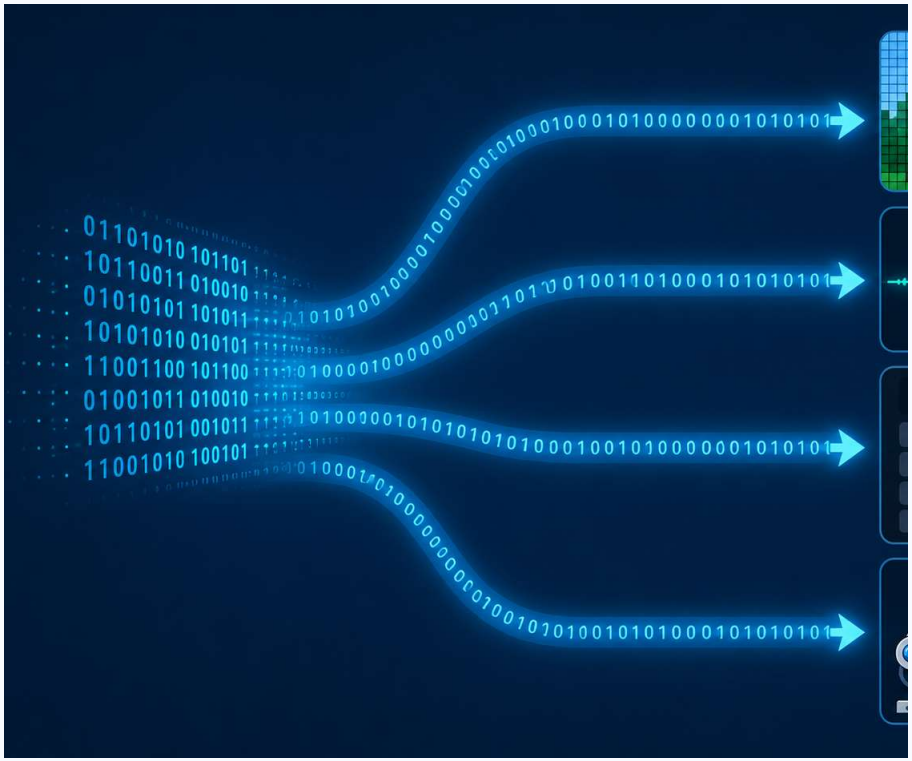
Phone camera
Light → pixels → image processing

Traffic signal
Timers + sensors → safe switching

Medical monitor
Voltage from sensor → heart-rate number

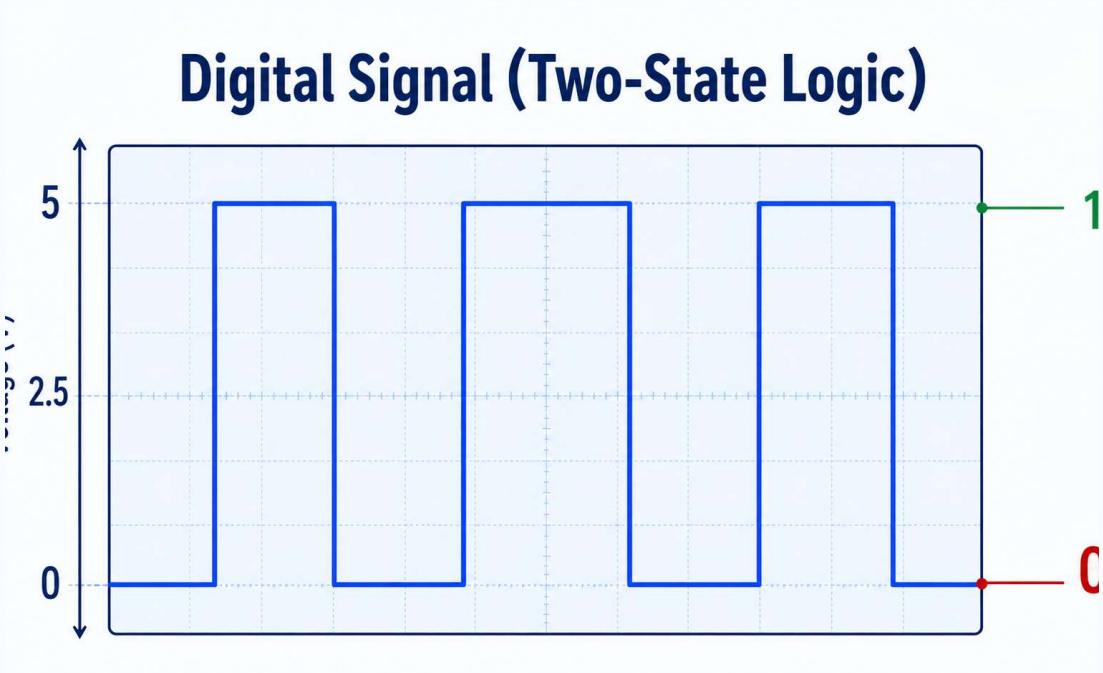
Robot / drone
Sensor data → control decisions

Digital payments
Bits represent account data securely



The key hardware idea: represent information using two reliable states

Real voltages are analog, but circuits deliberately interpret ranges as logic levels.



LOW range → bit 0
HIGH range → bit 1
Noise margin protects the decision

Why two states?
- Easy to build transistors as switches
- Robust against small voltage noise
- Simple memory: hold charge / no charge
- Boolean logic becomes arithmetic

From voltage levels to numbers

A sequence of bits can represent a count, a character, a sensor reading, an instruction, or a color.

8 bits = 1 byte

Unsigned number
178

ASCII-like character
one symbol

Sensor sample
temperature value

Instruction
operation code

Lecture map

By the end, students should be able to move comfortably among common number systems.

1. Why binary fits digital hardware
2. Decimal ↔ Binary
3. Binary ↔ Octal / Hexadecimal
4. Decimal ↔ Octal / Hexadecimal
5. 1’s and 2’s complements
6. Subtraction using complements
7. BCD and other binary codes

Learning habit
Always write the base: 1010_2 is not 1010_{10}.

Microcontroller link
Registers, ports and memory are naturally read in binary/hex.

Place value is the common idea behind every base

The base tells us which powers are used.

Base	Digits allowed	Place values
Decimal 10	0–9	… $10^3, 10^2, 10^1, 10^0$
Binary 2	0,1	… $2^3, 2^2, 2^1, 2^0$
Octal 8	0–7	… $8^3, 8^2, 8^1, 8^0$
Hex 16	0–9,A–F	… $16^3, 16^2, 16^1, 16^0$

Example

$$325_{10} = 3\times10^2 + 2\times10^1 + 5\times10^0$$

Binary follows the same rule

$$1011_2 = 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 = 11_{10}$$

Decimal to binary: repeated division by 2

Read the remainders from bottom to top.

Step	Quotient	Remainder
37 ÷ 2	18	1
18 ÷ 2	9	0
9 ÷ 2	4	1
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

$$37_{10} = 100101_2$$

Check using place values
$100101_2 = 32 + 4 + 1 = 37$

Practice seed
Convert 59_{10} to binary.

Binary to decimal: add the weighted 1-bits

Only positions carrying a 1 contribute to the sum.

Bit	1	1	0	1	0	1
Weight	32	16	8	4	2	1
Contribution	32	16	0	4	0	1

$$110101_2 = 32 + 16 + 4 + 1 = 53_{10}$$

Mental shortcut
List powers of 2: 1, 2, 4, 8, 16, 32, 64, 128…

Real numbers in binary: same idea, new point

A number like 321.78 has two parts:

321 . **78**

integer part fractional part

Left of point: 2^8 2^7 ... 2^1 2^0
Right of point: 2^{-1} 2^{-2} 2^{-3} ...

$$321.78_{10} \approx 101000001.110001111010_2$$

Many decimal fractions do not terminate in binary. In practical digital systems, we keep a fixed number of fractional bits and accept a tiny rounding error.

Decimal → Binary: integer part of 321.78_{10}

Step 1: Convert 321 by repeated division by 2.

Division		Quotient	Remainder
321 ÷ 2	→	160	1
160 ÷ 2	→	80	0
80 ÷ 2	→	40	0
40 ÷ 2	→	20	0
20 ÷ 2	→	10	0
10 ÷ 2	→	5	0
5 ÷ 2	→	2	1
2 ÷ 2	→	1	0
1 ÷ 2	→	0	1

Read remainders upward:

$$321_{10} = 101000001_{2}$$

The last remainder becomes the leftmost bit. This is why the remainder column is read from bottom to top.

Next: convert the fractional part .78.

Decimal fraction → Binary fraction: 0.78_{10}

Step 2: Multiply the fractional part by 2. The integer part each time becomes the next binary digit.

Step	Multiply by 2	Bit	New fraction
1	0.78 × 2 = 1.56	1	0.56
2	0.56 × 2 = 1.12	1	0.12
3	0.12 × 2 = 0.24	0	0.24
4	0.24 × 2 = 0.48	0	0.48
5	0.48 × 2 = 0.96	0	0.96
6	0.96 × 2 = 1.92	1	0.92
7	0.92 × 2 = 1.84	1	0.84
8	0.84 × 2 = 1.68	1	0.68
9	0.68 × 2 = 1.36	1	0.36
10	0.36 × 2 = 0.72	0	0.72
11	0.72 × 2 = 1.44	1	0.44
12	0.44 × 2 = 0.88	0	0.88

First 12 fractional bits:

$$.78_{10} \approx .110001111010_2$$

Therefore: $321.78_{10} \approx 101000001.110001111010_2$

With 12 fractional bits, the represented value is 321.77978515625_{10}, which is very close to 321.78.

Binary → Decimal: include bits after the point

Example: convert 101000001.110001111010_2 back to decimal.

Integer part

$$101000001_2 = 2^8 + 2^6 + 2^0$$
$$= 256 + 64 + 1$$
$$= 321$$

Fractional part

$$.110001111010_2$$
$$= 1\cdot2^{-1} + 1\cdot2^{-2} + 0\cdot2^{-3} + 0\cdot2^{-4} + 0\cdot2^{-5}$$
$$+ 1\cdot2^{-6} + 1\cdot2^{-7} + 1\cdot2^{-8} + 1\cdot2^{-9}$$
$$+ 0\cdot2^{-10} + 1\cdot2^{-11} + 0\cdot2^{-12}$$
$$= 0.77978515625$$

Final value:

$$321 + 0.77978515625$$
$$= 321.77978515625_{10}$$
$$\approx 321.78_{10}$$

Takeaway: bits after the binary point carry weights 1/2, 1/4, 1/8, 1/16, and so on.

Another binary → decimal real-number example

Convert 101101.1011_2 to decimal.

Integer side:

$101101_2 = 1\cdot2^5 + 0\cdot2^4 + 1\cdot2^3 + 1\cdot2^2 + 0\cdot2^1 + 1\cdot2^0$

$= 32 + 8 + 4 + 1$

$= 45$

Fraction side:

$.1011_2 = 1\cdot2^{-1} + 0\cdot2^{-2} + 1\cdot2^{-3} + 1\cdot2^{-4}$

$= 0.5 + 0.125 + 0.0625$

$= 0.6875$

$$101101.1011_2 = 45.6875_{10}$$

Binary ↔ octal and hexadecimal: group bits

Octal groups 3 bits. Hexadecimal groups 4 bits. Start grouping from the binary point; for integers, from the right.

Binary group	Octal	Hex
000	0	0
001	1	1
010	2	2
011	3	3
100	4	4
101	5	5
110	6	6
111	7	7

Binary group	Hex
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

Example
10110110_2
$= 10\ 110\ 110_2 = 266_8$
$= 1011\ 0110_2 = B6_{16}$

Worked conversions: binary, octal and hexadecimal

Each octal digit expands to 3 bits; each hex digit expands to 4 bits.

Binary → Hex
$1110\ 0101\ 1011_2 = E5B_{16}$

Binary → Octal
$110101011_2 = 110\ 101\ 011_2 = 653_8$

Hex → Binary
$3A_{16} = 0011\ 1010_2 = 111010_2$

Octal → Binary
$527_8 = 101\ 010\ 111_2 = 101010111_2$

Why hexadecimal matters in microcontrollers
A port, register or memory byte is often written as 0x3A instead of 00111010.

Decimal ↔ octal and hexadecimal

You can convert directly by division, or go through binary.

Decimal → Octal	Quotient	Remainder
156 ÷ 8	19	4
19 ÷ 8	2	3
2 ÷ 8	0	2

$$156_{10} = 234_{8}$$

Decimal → Hex	Quotient	Remainder
156 ÷ 16	9	12 = C
9 ÷ 16	0	9

$$156_{10} = 9C_{16}$$

Reverse check
$234_8 = 2\times64 + 3\times8 + 4 = 156$
$9C_{16} = 9\times16 + 12 = 156$

1’s complement and 2’s complement

Complements give a convenient way to represent negative numbers and perform subtraction using addition.

Given 8-bit number
A = 0010 1101_2

1’s complement
Flip every bit
1101 0010_2

2’s complement
1’s complement + 1
1101 0011_2

System	Positive number	Negative of that number
Sign-magnitude	00101101	10101101
1’s complement	00101101	11010010
2’s complement	00101101	11010011

Subtraction using 1’s complement: case A < B

Example: 9 − 25 using 8 bits. No end-around carry means the answer is negative.

Quantity	8-bit binary
A = 9	0000 1001
B = 25	0001 1001
1’s comp(B)	1110 0110
A + 1’s comp(B)	1110 1111
No carry → take 1’s comp	0001 0000

Magnitude = 16
Result = −16

Rule when no carry appears
Complement the sum to get magnitude, then attach a negative sign.

Subtraction using 2’s complement

Compute A − B as A + 2’s complement(B). Ignore any final carry.

Case	Work	Interpretation
25 − 9	00011001 + 11110111 = 1 00010000	Ignore carry → +16
9 − 25	00001001 + 11100111 = 11110000	MSB 1 → negative; 2’s comp = 00010000 → −16

Why this is cleaner than 1’s complement
No end-around carry rule. The same adder circuit can do addition and subtraction.

Hardware idea
Subtraction becomes: invert B, add 1, then add to A.

Quick classroom examples

Use these for board practice or clicker questions.

Question	Answer
Convert 45_{10} to binary	101101_2
Convert 101011_2 to decimal	43_{10}
Convert $7D_{16}$ to binary	$0111\ 1101_2$
Convert 110110101_2 to octal	665_8
8-bit 2’s complement of 0001 0100	1110 1100
8-bit value of 1110 1100 in 2’s complement	−20

Teaching tip
Ask students to write the base on every answer. It prevents silent mistakes.

Binary Coded Decimal (BCD)

BCD stores each decimal digit separately using 4 bits. It is different from pure binary.

Decimal digit	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Example
259_{10} in BCD
= 0010 0101 1001

But pure binary is
$259_{10} = 1\ 0000\ 0011_2$

Why use BCD?
Useful when decimal display or exact decimal digits matter, such as calculators, clocks and financial-style entry.

Other useful binary codes

Codes assign meanings to bit patterns for specific goals.

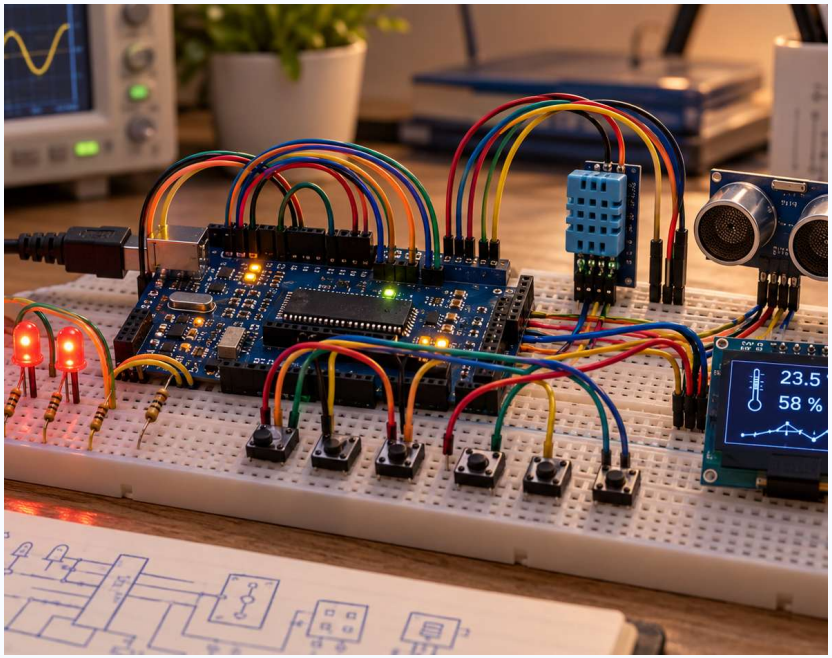
Code	Main idea	Common use
8421 BCD	Weighted digit code	Decimal displays
Excess-3	BCD digit + 3	Decimal arithmetic teaching / older systems
Gray code	Adjacent values differ by one bit	Rotary encoders, error reduction
ASCII / Unicode	Bits represent characters	Text storage and communication

Gray code example
Binary: 00, 01, 10, 11
Gray: 00, 01, 11, 10

ASCII idea
“A” can be stored as a bit pattern, just like a number is.

Bridge to microcontrollers: why binary and hex will keep appearing

Ports, registers and memory are organized as bit fields.



Example register byte
PORTB = 0b10110010
= 0xB2

Bit-level control
Each bit may turn an LED on, read a button, select a timer mode, or enable an interrupt.

Next lecture preview
Logic gates and Boolean algebra: how bits are processed.

Recap

Students should leave with these takeaways.

- Binary fits two-state electronic hardware.
- Place value works the same way in every base.
- Binary ↔ octal groups 3 bits; binary ↔ hex groups 4 bits.
- 2’s complement is the standard practical way to represent signed integers.
- BCD and other codes assign special meaning to bit patterns.

One-line summary
Digital systems do not merely store 0s and 1s — they assign meaning to bit patterns and process them predictably.

Suggested practice

A short exercise set for after Lecture 1.

#	Task
1	Convert 73_{10} to binary, octal and hexadecimal.
2	Convert 101101101_2 to decimal, octal and hexadecimal.
3	Find the 1's and 2's complement of 01011010_2.
4	Using 1's complement subtraction: compute 42 − 18 and 18 − 42.
5	Using 2's complement subtraction: compute 42 − 18 and 18 − 42.
6	Write 406_{10} in BCD and compare it with pure binary.

Optional lab warmup
Look at a microcontroller GPIO register and identify what each bit controls.